Assignment - 1

Simulation and Analysis of DoS Attacks using Python and Wireshark

Network Security

Submitted by-

Anchita Sharma

500120586

Batch – 4 [CSF]

1. Introduction

Denial of Service (DoS) attacks are cyber-attacks that attempt to make a network service or system unavailable by overwhelming it with a large amount of traffic. The attacker sends excessive requests to consume the resources of the target system, which prevents legitimate users from accessing the service.

In this experiment, two types of DoS attacks were simulated using Python and the Scapy library: **TCP SYN Flood Attack** and **ICMP Flood Attack**. The generated attack traffic was captured and analyzed using **Wireshark**, a network packet analysis tool. The aim of the experiment was to observe abnormal traffic patterns produced during these attacks and analyze them through packet inspection.

2. Objective

The objectives of this experiment are:

- To simulate DoS attacks using Python.
- To generate malicious network traffic using the Scapy library.
- To capture attack traffic using Wireshark.
- To analyze the behavior of SYN Flood and ICMP Flood attacks.
- To understand how abnormal packet patterns appear during network attacks.

3. Tools Used

Tool	Purpose
Python	Used to create attack scripts
Scapy Library	Used to generate custom network packets
Wireshark	Used to capture and analyze network traffic
PyScripter	Used to write and execute Python scripts

4. SYN Flood Attack

4.1 Description

The SYN Flood attack is a type of Denial-of-Service attack that targets the **TCP three-way handshake process**. Normally, a TCP connection is established using the following sequence:

Client → SYN
Server → SYN-ACK
Client → ACK

In a SYN Flood attack, the attacker sends a large number of **SYN packets** to the target server but does not complete the handshake process. As a result, the server allocates resources for each half-open connection and waits for the final ACK packet. When many such incomplete connections are created, the server resources become exhausted, leading to service disruption.

In this experiment, a Python script using the Scapy library was used to generate multiple SYN packets to simulate the attack.

4.2 Implementation

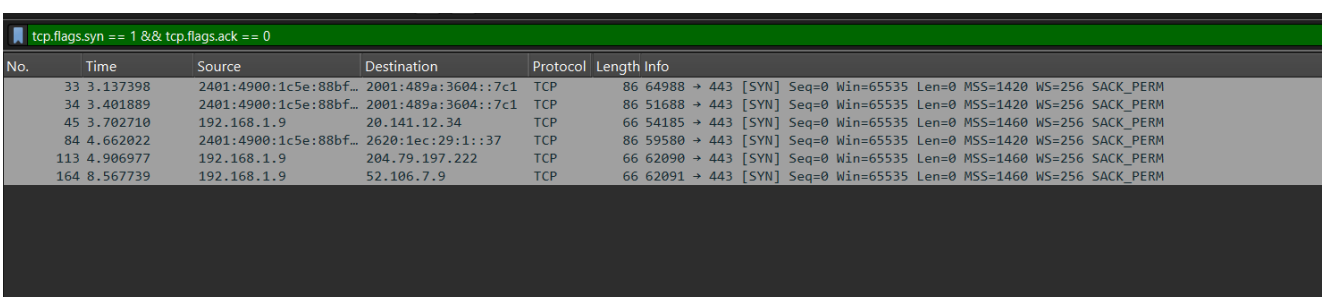
The SYN Flood attack was implemented using Python and Scapy. The script sends multiple TCP packets with the SYN flag set to initiate connection requests to the target IP address.

The code generates packets with random source ports and sends them repeatedly to simulate multiple connection attempts.

4.3 Wireshark Analysis

Wireshark was used to capture the network packets generated during the SYN Flood attack. The following filter was applied to observe SYN packets:

tcp.flags.syn == 1 && tcp.flags.ack == 0



tcp.flags.syn == 1 && tcp.flags.ack == 0

No.	Time	Source	Destination	Protocol	Length	Info
33	3.137398	2401:4900:1c5e:88bf…	2001:489a:3604::7c1	TCP	86	64988 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1420 WS=256 SACK_PERM
34	3.401889	2401:4900:1c5e:88bf…	2001:489a:3604::7c1	TCP	86	51688 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1420 WS=256 SACK_PERM
45	3.702710	192.168.1.9	20.141.12.34	TCP	66	54185 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
84	4.662022	2401:4900:1c5e:88bf…	2620:1ec:29:1::37	TCP	86	59580 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1420 WS=256 SACK_PERM
113	4.906977	192.168.1.9	204.79.197.222	TCP	66	62090 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
164	8.567739	192.168.1.9	52.106.7.9	TCP	66	62091 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM

Figure 1: Wireshark capture showing multiple TCP SYN packets.

This filter displays TCP packets where the SYN flag is set but the ACK flag is not set, indicating connection requests that have not yet been acknowledged.

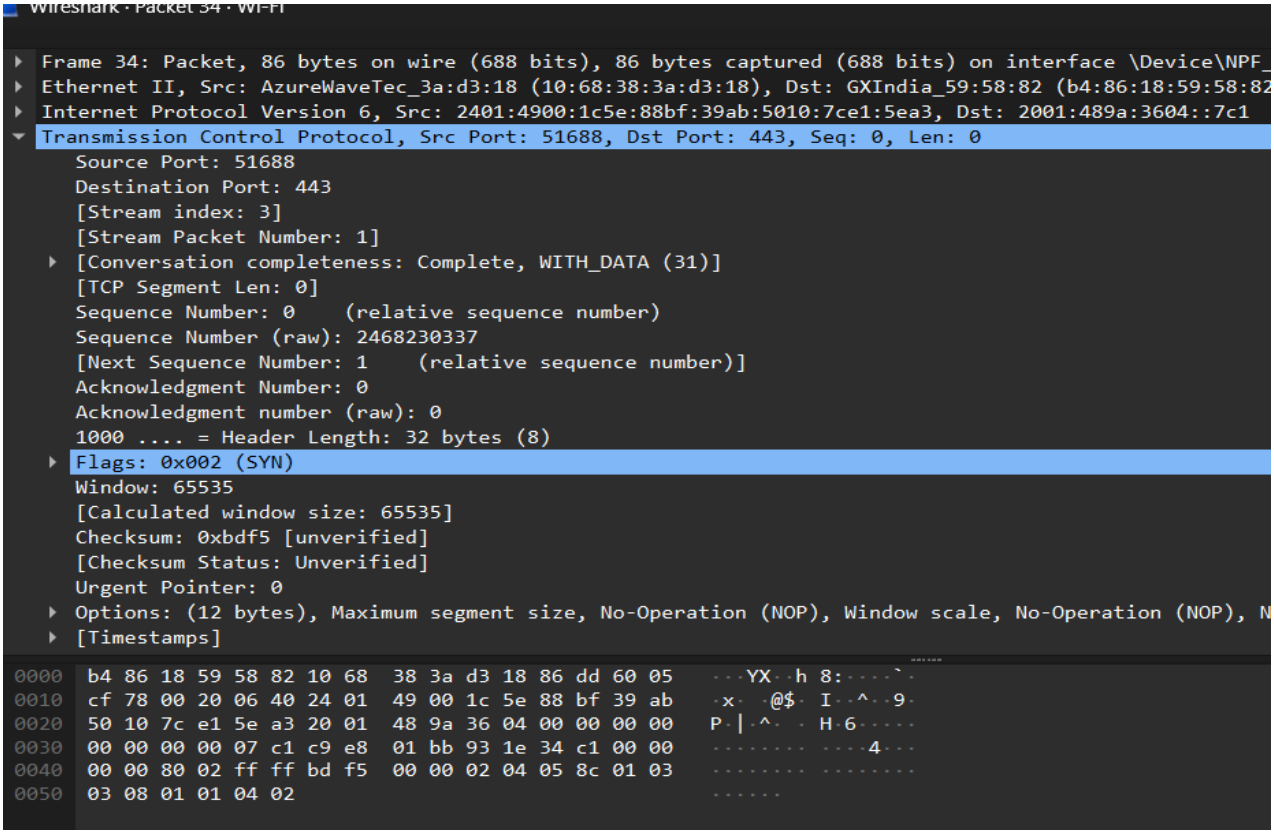
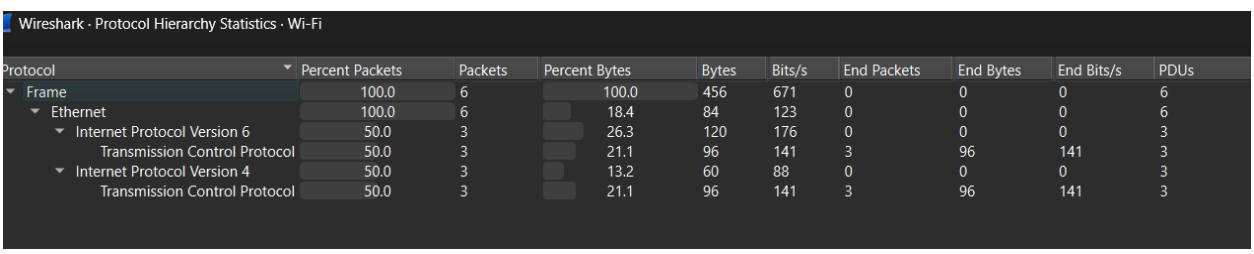


Figure 2: TCP packet details showing SYN flag.

The captured packets show multiple SYN packets being sent from the attacker machine to the destination server. This pattern represents repeated connection attempts typical of a SYN Flood attack.



Wireshark · Protocol Hierarchy Statistics · Wi-Fi

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	6	100.0	456	671	0	0	0	6
Ethernet	100.0	6	18.4	84	123	0	0	0	6
Internet Protocol Version 6	50.0	3	26.3	120	176	0	0	0	3
Transmission Control Protocol	50.0	3	21.1	96	141	3	96	141	3
Internet Protocol Version 4	50.0	3	13.2	60	88	0	0	0	3
Transmission Control Protocol	50.0	3	21.1	96	141	3	96	141	3

Figure 3: Wireshark protocol hierarchy statistics during SYN Flood attack.

The packet details also show the TCP header containing:

- Source port
- Destination port
- Sequence number
- SYN flag

These details confirm that multiple TCP connection requests were generated during the attack simulation.

5. ICMP Flood Attack

5.1 Description

The ICMP Flood attack is another type of Denial-of-Service attack that targets the network bandwidth of a system. It involves sending a large number of **ICMP Echo Request packets** (commonly known as ping requests) to the target system.

When the target receives a large number of ICMP requests, it attempts to process and respond to each request. This consumes system resources and network bandwidth, which can degrade system performance or make the service unavailable to legitimate users.

5.2 Implementation

In this experiment, an ICMP Flood attack was simulated using Python and the Scapy library. The script repeatedly sends ICMP Echo Request packets to the target IP address.

Each packet contains an ICMP header and an IP header, and the script sends hundreds of such packets in quick succession to simulate traffic flooding.

5.3 Wireshark Analysis

Wireshark was used to capture and analyze the ICMP packets generated by the attack script. The following filter was applied to display ICMP traffic:

Icmp

icmp

No.	Time	Source	Destination	Protocol	Length	Info
40	6.953753	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
41	6.959181	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
42	6.964071	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
43	6.969947	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
44	6.975059	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
45	6.979228	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
46	6.983431	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
47	6.990738	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
48	6.995253	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
49	6.999127	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
50	7.004515	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
51	7.010389	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
52	7.016885	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
53	7.024255	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
54	7.029469	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
55	7.035236	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
56	7.041128	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
57	7.047648	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
58	7.052550	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
59	7.057835	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
60	7.066660	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
61	7.070864	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
62	7.079715	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
63	7.085369	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
64	7.089976	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
65	7.095971	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
66	7.101063	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
67	7.106971	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
68	7.111106	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)
69	7.115693	192.168.1.9	8.8.8.8	ICMP	42	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (no response found!)

Figure 4: Wireshark capture showing ICMP Echo Request packets.

The captured packets show multiple icmp Echo Request packets originating from the attacking machine.

In the packet details section of Wireshark, the following information was observed:

- Internet Control Message Protocol
- Type: 8 (Echo Request)
- Code: 0

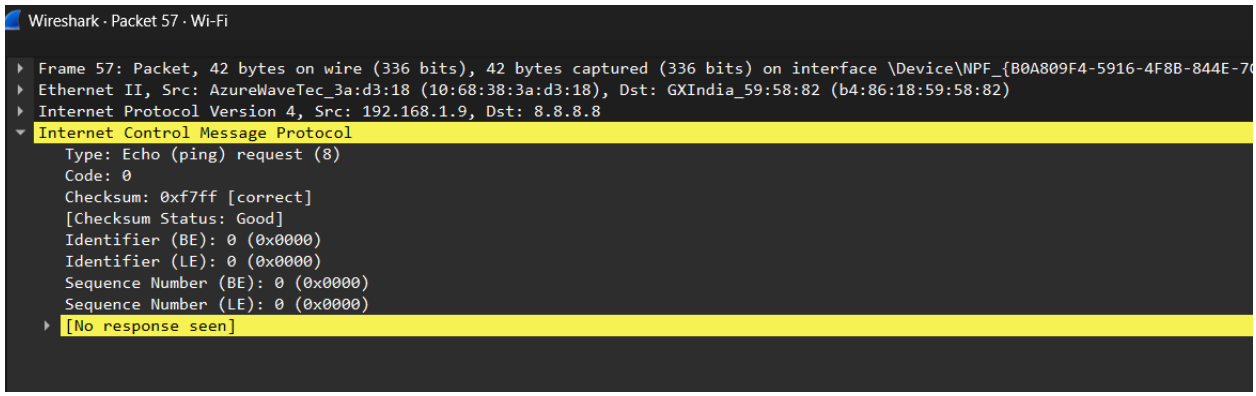
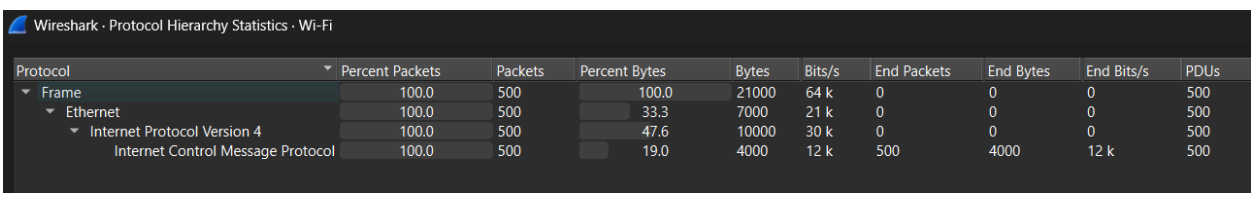


Figure 5: ICMP packet details showing Echo Request message.

These values indicate that the packets are ICMP Echo Requests generated during the attack simulation.



Wireshark · Protocol Hierarchy Statistics · Wi-Fi

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	500	100.0	21000	64 k	0	0	0	500
Ethernet	100.0	500	33.3	7000	21 k	0	0	0	500
Internet Protocol Version 4	100.0	500	47.6	10000	30 k	0	0	0	500
Internet Control Message Protocol	100.0	500	19.0	4000	12 k	500	4000	12 k	500

Figure 6: Protocol hierarchy statistics during ICMP Flood attack.

In some cases, Echo Reply packets were not observed due to network filtering or firewall restrictions. However, the repeated Echo Request packets confirm the generation of ICMP flood traffic.

6. Results

The experiment successfully demonstrated the simulation of two Denial of Service attacks: SYN Flood and ICMP Flood. Using Python and Scapy, a large number of packets were generated to simulate malicious traffic. Wireshark effectively captured and displayed the packet data, allowing detailed analysis of packet headers and protocol behavior.

The captured traffic showed repeated SYN packets in the SYN Flood attack and repeated ICMP Echo Request packets in the ICMP Flood attack, confirming the presence of attack patterns.

7. Conclusion

This experiment demonstrated how Denial of Service attacks can generate abnormal network traffic patterns that disrupt normal network operations. By using Wireshark to analyze packet data, it is possible to identify suspicious traffic such as repeated SYN requests or excessive ICMP packets.

Understanding these traffic patterns is important for detecting and mitigating network attacks in real-world systems. Tools like Wireshark and Scapy are valuable for studying network behavior and improving cybersecurity awareness.